

SIMATS ENGINEERING

ASSESSMENT TOOL 1

Experiments

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1153

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models. (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:4

Annexure A

1. (Reg. No. 192311051)

Total Marks:50

Experiments

a) Develop a system using UML for implementing the Book Bank process. It should verify the details of the student by the central computer. The details regarding the student will be provided to the central computer through the administrator in the book bank and the computer will verify the details of the student and provide approval to the office. Then the books that are needed by the student will issue from the office to him.

b) Develop a system using UML for implementing the Exam Registration Portal. It should verify the details of the candidate by the central computer. The details regarding the candidate will be provided to the central computer through the administrator and the computer will verify the details of the candidate and provide approval. Then the hall ticket will be issued from the office to the candidate.

Experiment (a): Book Bank Process

Title

UML Design for Book Bank Management System

Aim

To develop a UML-based Book Bank Management System that verifies student details through a central computer and issues books to eligible students.

Problem Description

The Book Bank System allows students to request books. The administrator enters student details into the system. The central computer verifies the details and provides approval to the office. After approval, the required books are issued to the student.

Actors

1. Student
 2. Administrator
 3. Office Staff
 4. Central Computer System
-

Use Cases

Student

- Submit Book Request
- Receive Books

Administrator

- Enter Student Details
- Update Student Details
- View Reports

Central Computer

- Verify Student Details
- Approve Request

Office Staff

- Issue Books

Relationships

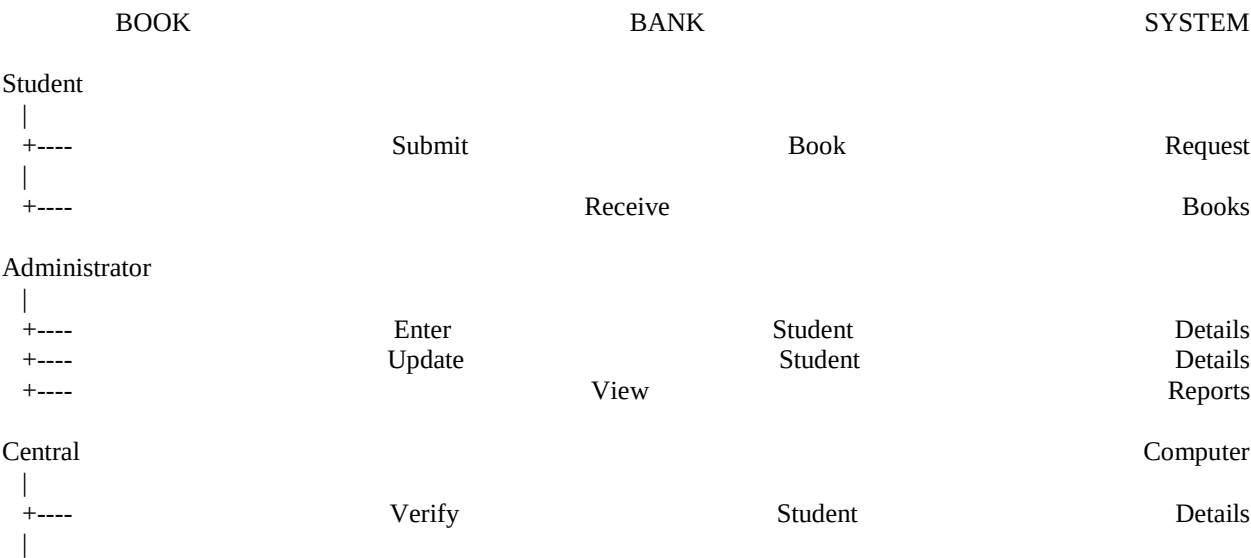
<<include>>

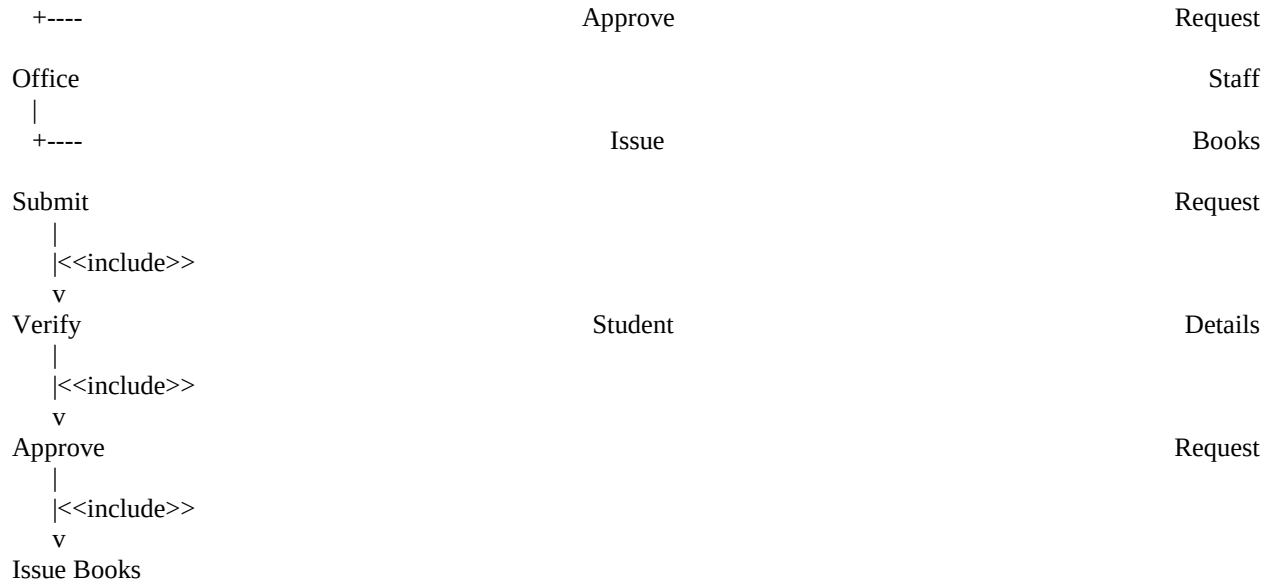
- Submit Book Request <<include>> Verify Student Details
- Verify Student Details <<include>> Approve Request
- Issue Books <<include>> Approve Request

Association

- Student ↔ Submit Book Request
- Administrator ↔ Enter Student Details
- Office Staff ↔ Issue Books
- Central Computer ↔ Verify Student Details

Use Case Diagram (Text)





Class Diagram

Student

- studentID
- name
- department
- year

Methods:

- requestBook()
- viewStatus()

Administrator

- adminID
- name

Methods:

- enterDetails()
- updateDetails()

Book

- bookID
- title

- author

Methods:

- issueBook()
- returnBook()

Office

- officeID

Methods:

- approveIssue()
- issueBooks()

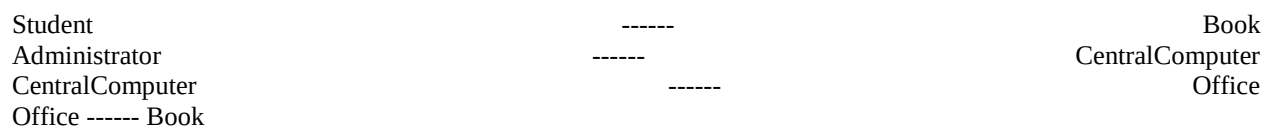
CentralComputer

- systemID

Methods:

- verifyStudent()
- approveRequest()

Relationships



Conclusion

The Book Bank Management System was successfully modeled using UML. The system verifies student details through the central computer, approves eligible requests, and enables the office to issue books efficiently.

Experiment (b): Exam Registration Portal

Title

UML Design for Exam Registration Portal

Aim

To develop a UML-based Exam Registration Portal that verifies candidate details through a central computer and issues hall tickets to eligible candidates.

Problem Description

The candidate registers for the examination through the portal. The administrator enters and manages candidate details. The central computer verifies the details and grants approval. Once approved, the office generates and issues the hall ticket.

Actors

1. Candidate
 2. Administrator
 3. Office Staff
 4. Central Computer System
-

Use Cases

Candidate

- Register for Exam
- Receive Hall Ticket

Administrator

- Enter Candidate Details

- Update Candidate Details
- View Reports

Central Computer

- Verify Candidate Details
- Approve Registration

Office Staff

- Generate Hall Ticket
- Issue Hall Ticket

Relationships

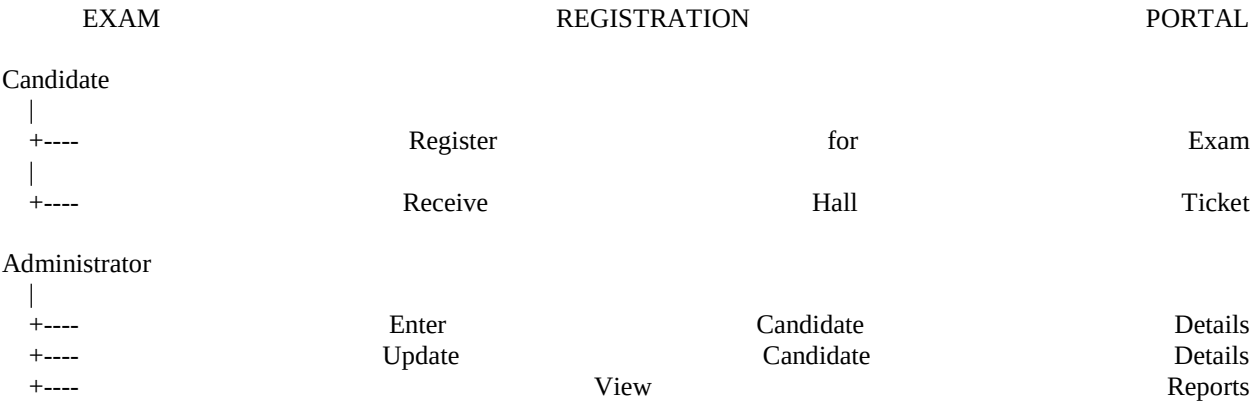
<<include>>

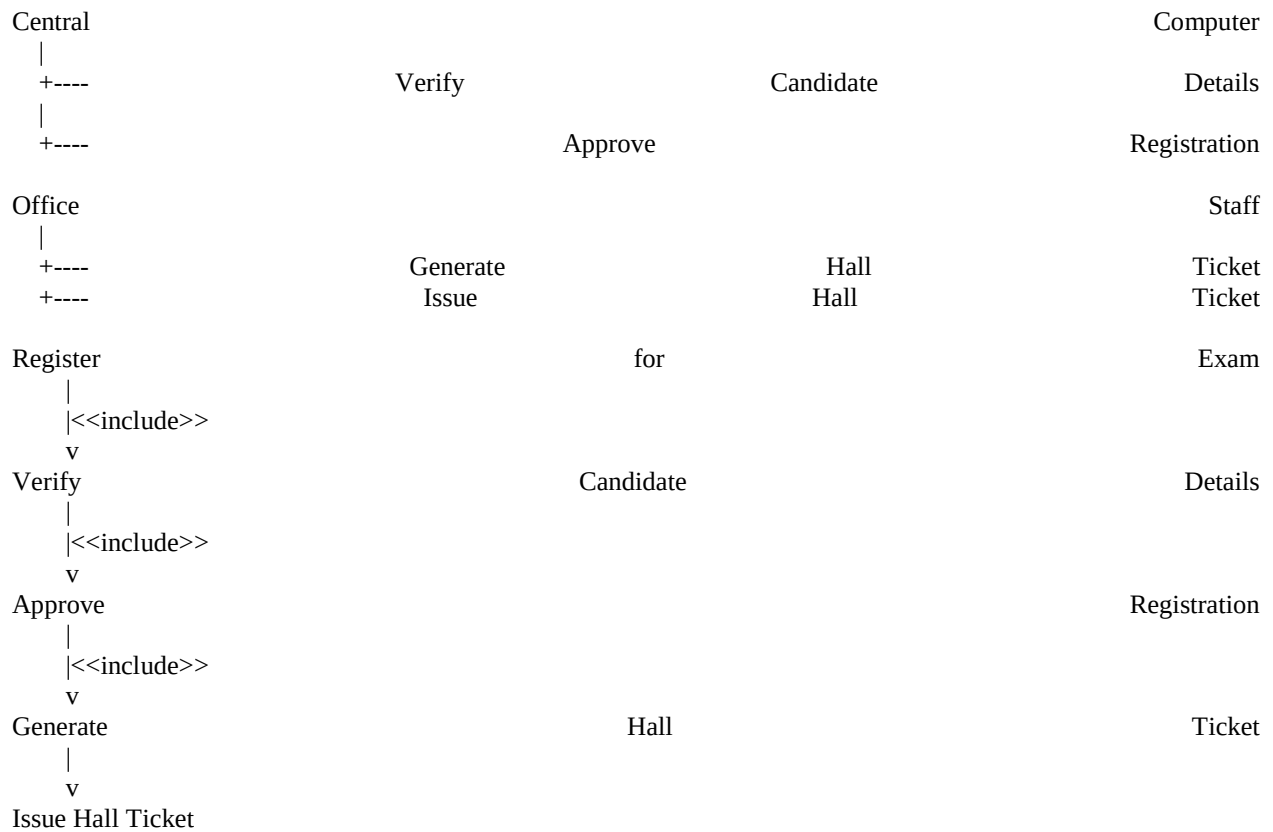
- Register for Exam <<include>> Verify Candidate Details
- Verify Candidate Details <<include>> Approve Registration
- Generate Hall Ticket <<include>> Approve Registration

Association

- Candidate ↔ Register for Exam
- Administrator ↔ Enter Candidate Details
- Office Staff ↔ Generate Hall Ticket
- Central Computer ↔ Verify Candidate Details

Use Case Diagram (Text)





Class Diagram

Candidate

- candidateID
- name
- department
- semester

Methods:

- registerExam()
- downloadHallTicket()

Administrator

- adminID
- name

Methods:

- enterCandidateDetails()
- updateDetails()

HallTicket

- hallTicketNo
- examDate
- venue

Methods:

- generateHallTicket()
- printHallTicket()

Office

- officeID

Methods:

- issueHallTicket()

CentralComputer

- systemID

Methods:

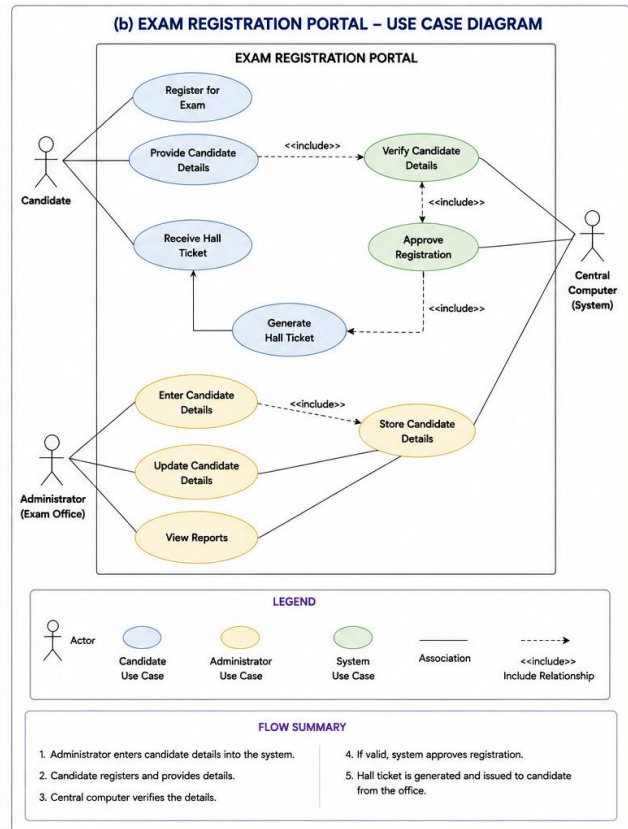
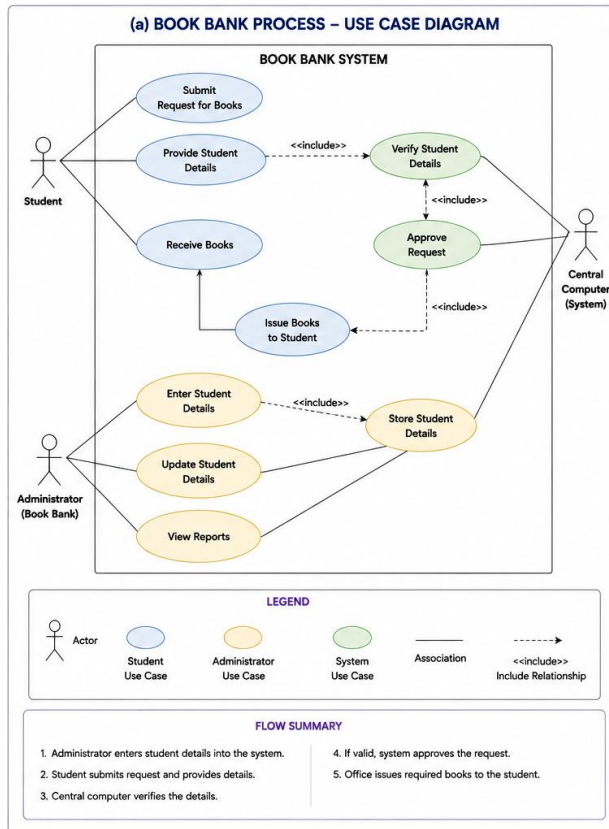
- verifyCandidate()
- approveRegistration()

Relationships

Candidate	-----	HallTicket
Administrator	-----	CentralComputer
CentralComputer	-----	Office
Office	-----	HallTicket

Conclusion

The Exam Registration Portal was successfully modeled using UML. The system verifies candidate details through a central computer, approves eligible registrations, and generates hall tickets efficiently, ensuring a streamlined examination process.



Code of Conduct:

I D. Raghu Vardhan-192311051(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D. Raghu Vardhan

RUBRICS FOR CALCULATION:**Case Study**

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation